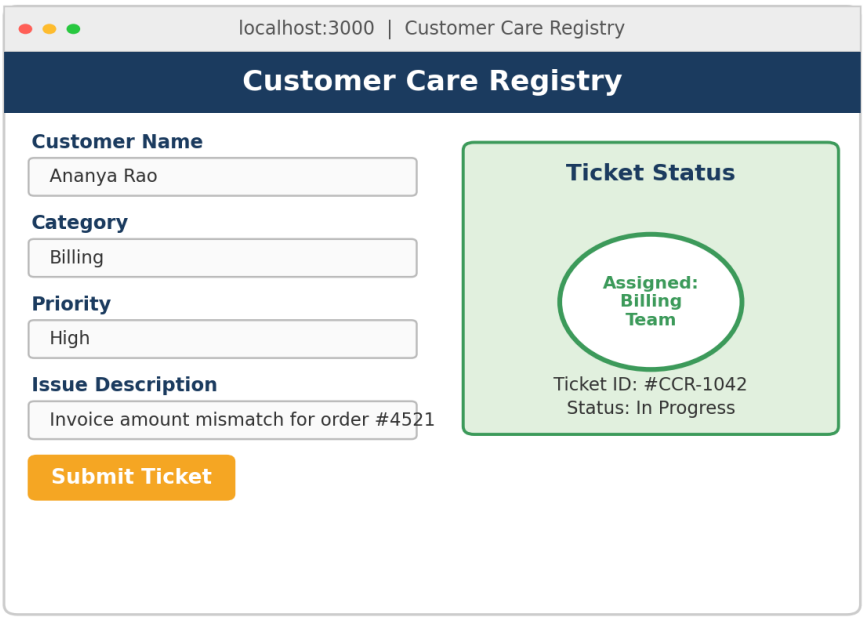
FINAL PROJECT REPORT

Customer Care Registry System using MERN Stack

A Centralized Platform for Managing Customer Interactions, Issues and Feedback



Domain	Web Development / Full-Stack Application
Core Technologies	MongoDB, Express.js, React.js, Node.js (MERN)
Supporting Tools	JWT Auth, Mongoose, Axios, Bootstrap, Nodemailer
Deployment	Render / Vercel with MongoDB Atlas
Environment	VS Code, Postman, npm, Git & GitHub

Table of Contents

1. Introduction

1.1 Project Overview

A Customer Care Registry is a centralized system that records and manages customer interactions, issues, and feedback. It enables businesses to streamline support processes, track inquiries, and analyse trends to enhance service quality. By maintaining a comprehensive history of customer interactions, the registry ensures consistency in responses, identifies recurring pain points, and aids in proactive issue resolution.

With data-driven insights, businesses can refine training programs, optimise service protocols, and offer personalised support, ultimately improving customer satisfaction and loyalty. In a competitive landscape, a Customer Care Registry serves as a crucial tool for fostering trust, efficiency, and long-term customer relationships.

This project builds a full-stack web application using the MERN stack (MongoDB, Express.js, React.js, Node.js) that allows customers to raise support tickets and allows agents and managers to track, assign, and resolve them through a single, centralised dashboard, replacing scattered emails, calls and spreadsheets.

1.2 Purpose

The Customer Care Registry was developed to give businesses a single source of truth for every customer interaction, instead of relying on fragmented channels such as email threads, phone logs and manual spreadsheets. It brings structure and accountability to the support process by giving every issue a traceable lifecycle from submission to resolution.

The purpose of this project is to:

- Provide a centralised, searchable record of every customer complaint, request and piece of feedback.
- Automate ticket assignment, status tracking and escalation so no issue is left unattended.
- Give managers real-time visibility into support workload, response times and customer satisfaction.
- Demonstrate an end-to-end MERN stack application — from database design and REST API development to a deployed, responsive React front end.

2. Ideation Phase

2.1 Problem Statement

Businesses of every size depend on responsive customer support to retain trust and loyalty. However, tracking customer issues manually across emails, phone calls and spreadsheets is time-consuming, inconsistent, and does not scale as the volume of complaints grows. There is a need for a centralised, easy-to-use system that logs every interaction, assigns it to the right agent, and gives managers visibility into open, in-progress and resolved issues.

Problem Statement Template	
I am (user)	a support agent / manager / customer interacting with a business
I'm trying to	log, track and resolve customer issues quickly and consistently
But	manual tracking across emails, calls and spreadsheets is slow and error-prone
Because	there is no single, centralised record of a customer's interaction history
Which makes me feel	frustrated by duplicate work, missed follow-ups and inconsistent responses

2.2 Empathy Map Canvas

The empathy map below captures the perspective of a typical end user (a support agent or manager) of the system.

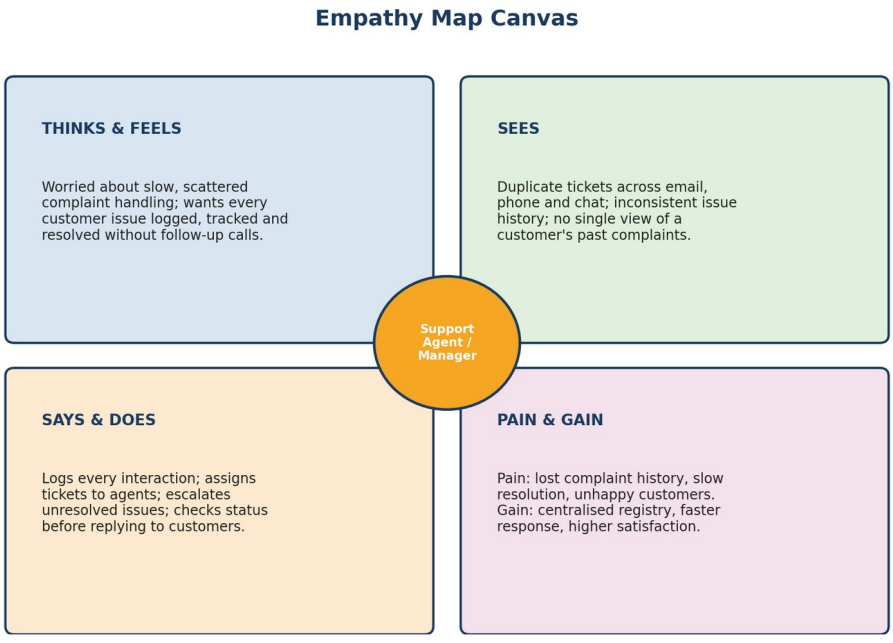


Figure 2.1: Empathy Map Canvas for the primary user persona

2.3 Brainstorming

The team explored multiple approaches before finalising the solution design. Key ideas generated during brainstorming include:

- Build a simple spreadsheet-based tracker shared across the support team.
- Develop a full-stack MERN web application with a REST API and a centralised MongoDB database.

- Add role-based access so customers, agents and managers see only what is relevant to them.
- Provide a real-time dashboard with ticket status, category and priority analytics.
- Send automated email notifications on ticket creation, assignment and resolution.
- Allow customers to rate the resolution and leave feedback on each closed ticket.
- Enable live chat / chatbot support for instant queries (future enhancement).

After evaluation, the team selected the MERN stack web application with a REST API and role-based dashboards, as it offered the best balance of scalability, real-time visibility, and ease of deployment.

3. Requirement Analysis

3.1 Customer Journey Map

The journey map below illustrates the end-to-end experience of a customer interacting with the Customer Care Registry, from raising an issue to receiving feedback confirmation.

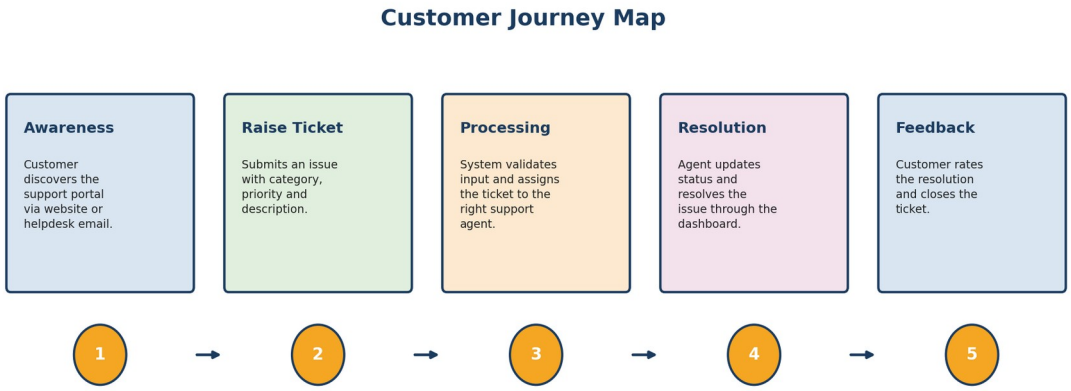


Figure 3.1: Customer Journey Map

3.2 Solution Requirement

Functional Requirements

- The system shall allow customers to register, log in, and submit a support ticket with category, priority and description.
- The system shall validate and store ticket data in a MongoDB database via a REST API.
- The system shall allow agents to view, accept, and update the status of assigned tickets.
- The system shall allow managers to view all tickets, reassign agents, and monitor overall performance through a dashboard.
- The system shall notify customers by email when their ticket status changes.

Non-Functional Requirements

- **Usability:** The interface should be simple and intuitive for non-technical customers and agents alike.
- **Performance:** API responses should be returned within 1-2 seconds under normal load.
- **Security:** Authentication and authorisation should be handled using JWT and hashed passwords (bcrypt).
- **Scalability:** The architecture should support adding new ticket categories or scaling to more concurrent users.
- **Portability:** The application should be deployable on any cloud platform supporting Node.js and MongoDB Atlas.

3.3 Data Flow Diagram

The Data Flow Diagram (Level 1) below shows how data moves through the system, from ticket submission to assignment, storage, and resolution tracking.

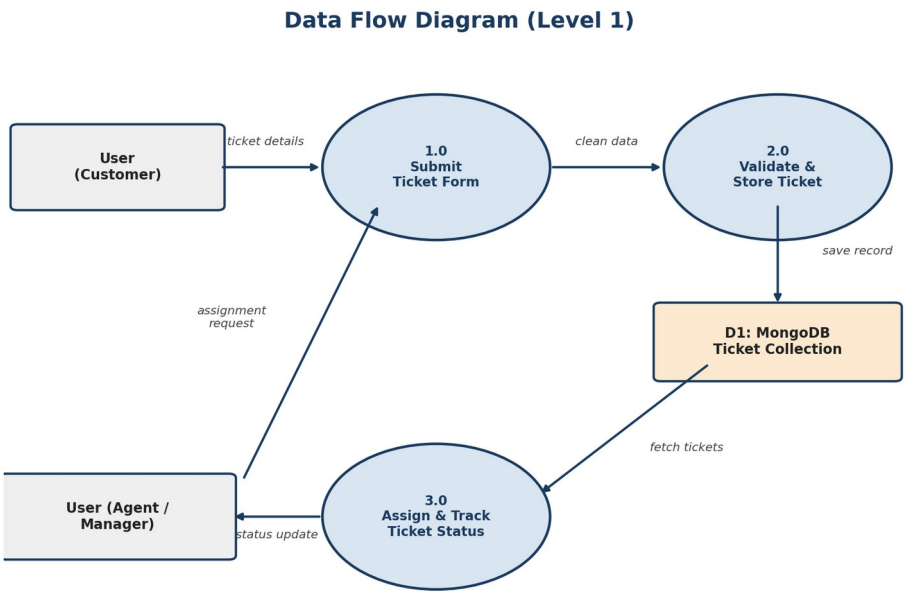


Figure 3.2: Data Flow Diagram (Level 1)

3.4 Technology Stack

Category	Technology / Tool
Frontend	React.js, React Router, Axios, Bootstrap / Tailwind CSS
Backend / Runtime	Node.js, Express.js
Database	MongoDB (Mongoose ODM), MongoDB Atlas
Authentication	JSON Web Tokens (JWT), bcrypt password hashing
Notifications	Nodemailer (email alerts on ticket updates)
State Management	React Context API / Redux Toolkit
Deployment	Render / Vercel / Netlify
Version Control	Git & GitHub

Skills Required

Building and deploying this solution calls for the following skill set:

Skill	Skill
JavaScript (ES6+)	React.js
Node.js & Express.js	MongoDB & Mongoose
REST API Design	JWT Authentication
HTML5 & CSS3	Git & GitHub
Postman (API Testing)	Agile Project Management

4. Project Design

4.1 Problem Solution Fit

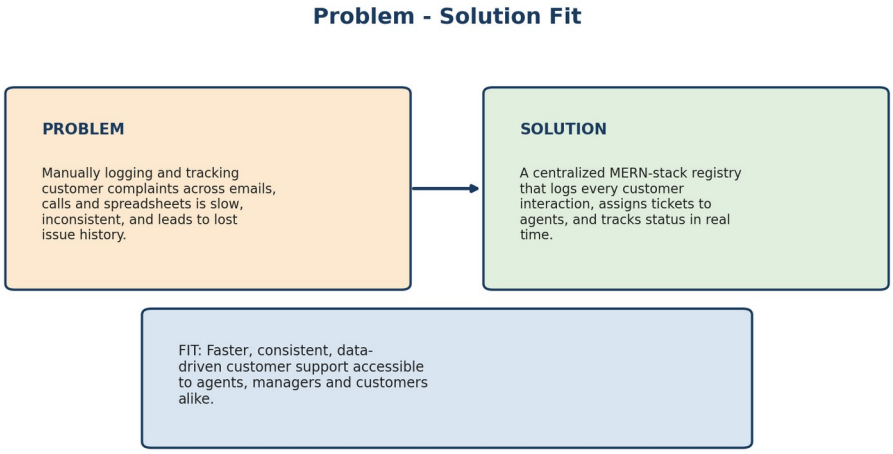


Figure 4.1: Problem - Solution Fit Canvas

4.2 Proposed Solution

The proposed solution is a web-based Customer Care Registry built on the MERN stack. Every customer interaction — a complaint, a request, or a piece of feedback — is captured as a ticket with a category, priority and status. Once submitted, the ticket flows through validation, assignment and resolution stages, with agents and managers interacting through role-based React dashboards backed by a secure Express REST API and a MongoDB database.

Aspect	Description
Data Source	Customer-submitted tickets captured through the React front-end form
Core Stack	MongoDB, Express.js, React.js, Node.js (MERN)
System Output	Ticket record with status (Open / In Progress / Resolved / Closed) and history
Deployment Mode	Node/Express REST API with a React single-page application front end
Target Users	Customers, support agents, support managers and administrators

Illustrative Usage Scenarios

The following scenarios illustrate how different users would interact with the deployed system and the kind of insight it delivers:

Scenario	Input / Trigger	System Behaviour
1. Raising a High- Priority Complaint	A customer submits a billing issue marked as High priority with a detailed description.	The system logs the ticket, notifies the billing team, and shows the customer a live status.
2. Tracking Ticket Aging	A manager reviews tickets that have remained Open for more than 48 hours.	The dashboard flags aging tickets so the manager can reassign or escalate them.
3. Closing an Issue with Feedback	An agent resolves a technical ticket and the customer confirms the fix.	The system prompts the customer for a satisfaction rating and closes the ticket.

4.3 Solution Architecture

The architecture below illustrates the complete flow of the system — from the React client through the Express REST API, authentication middleware, and business logic, down to the MongoDB Atlas database.

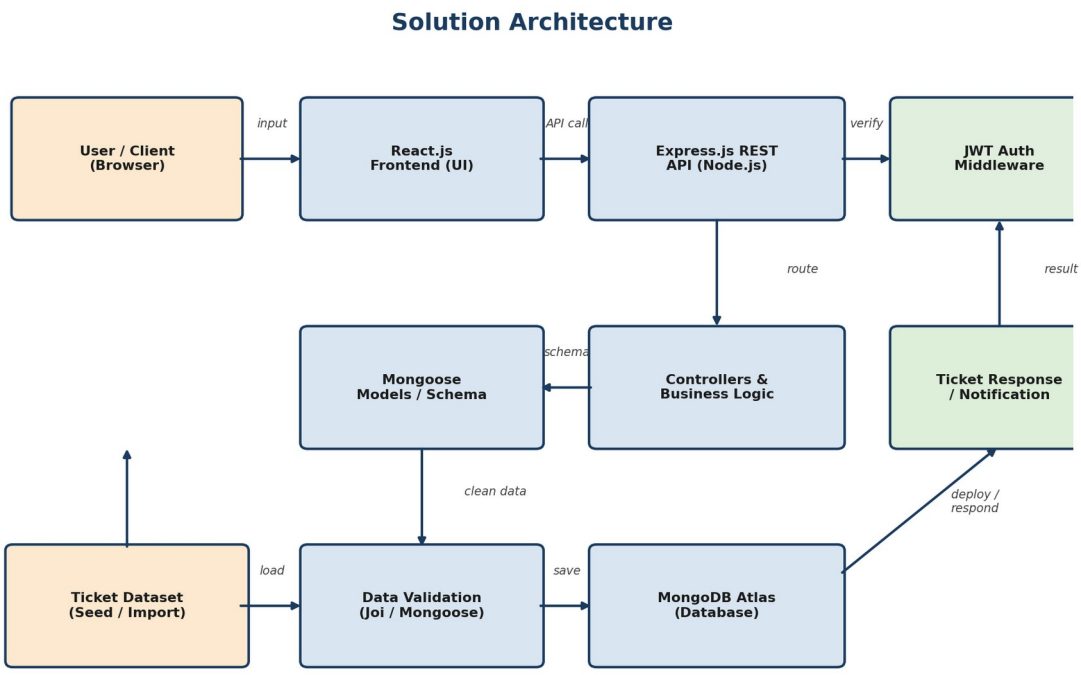


Figure 4.2: Solution Architecture Diagram

5. Project Planning & Scheduling

5.1 Project Planning

The project was executed in the following phases over the course of the development timeline:

Sprint	Task	Duration	Status
Sprint 1	Requirement gathering, wireframing & problem definition	Week 1	Completed
Sprint 2	Database design (MongoDB schema) & API planning	Week 2	Completed
Sprint 3	Backend development — Express REST API & JWT auth	Week 3	Completed
Sprint 4	Frontend development — React components & dashboards	Week 4	Completed
Sprint 5	Integration of frontend with backend API & notifications	Week 5	Completed
Sprint 6	Functional testing, performance testing & documentation	Week 6	Completed

An Agile methodology was followed, with tasks broken down into weekly sprints. Regular reviews were conducted at the end of each sprint to track progress against the project plan and adjust priorities as needed.

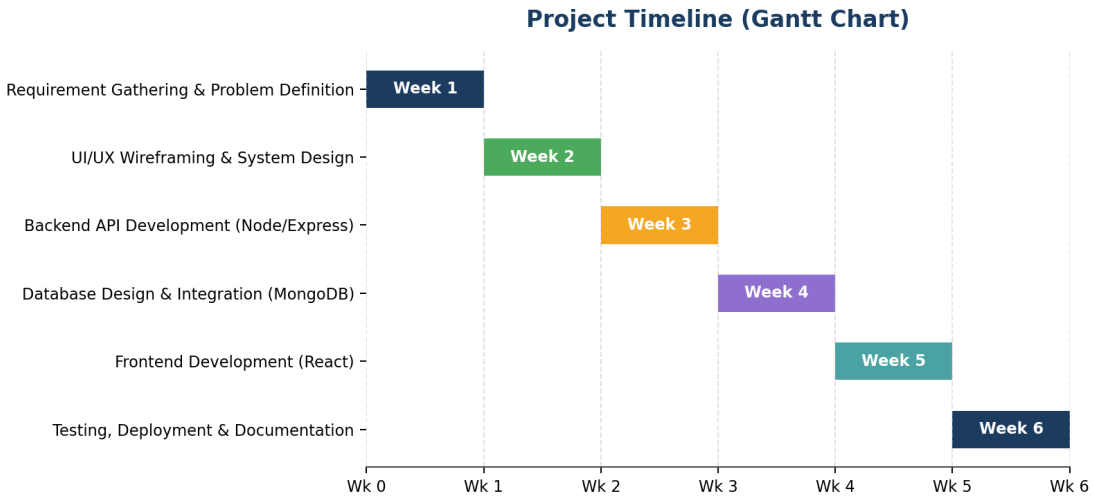


Figure 5.1: Project timeline (Gantt chart) across sprints

6. Functional and Performance Testing

Functional testing verified that each component of the application — from form input validation to ticket creation, assignment, and status updates — worked as intended.

Test Case	Expected Result	Status
Register / login with valid credentials	JWT token issued, user redirected to dashboard	Pass
Submit ticket with blank / missing field	Validation error message shown	Pass
Submit ticket with valid data	Ticket created and visible in agent dashboard	Pass
Access agent route as unauthenticated user	Request rejected with 401 Unauthorized	Pass
Update ticket status as agent	Status change reflected instantly on customer view	Pass
Cross-browser rendering	UI displays correctly on Chrome, Firefox, Edge	Pass

6.1 Performance Testing

API endpoints were benchmarked individually, and the application was load-tested with a growing number of concurrent users to evaluate response time and stability.

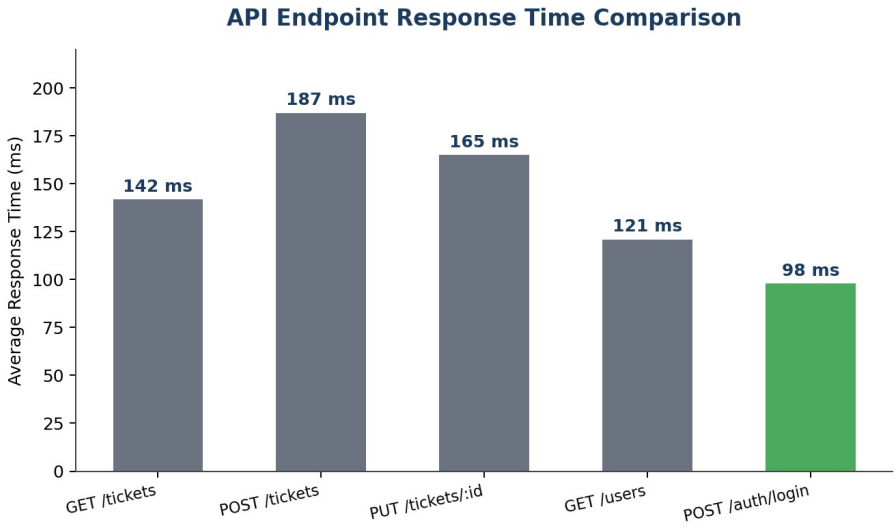


Figure 6.1: Average response time across key API endpoints

All endpoints returned responses within the 1-2 second performance requirement under normal load, with authentication and read-only endpoints performing fastest. Load test results, showing response time scaling with concurrent users, are presented in Section 7.

7. Results

7.1 Output Screenshots

The following visualisations and interface screenshots illustrate the ticket data analysis, system performance, and the final deployed application.

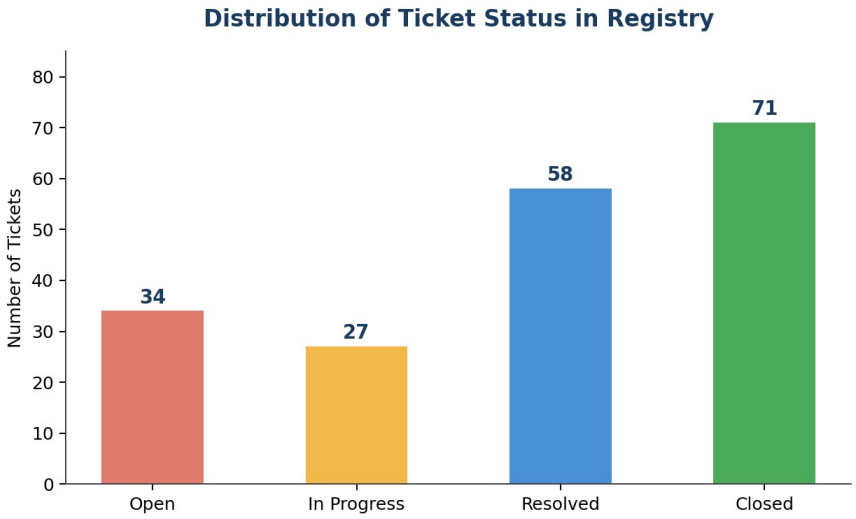


Figure 7.1: Distribution of ticket status in the registry

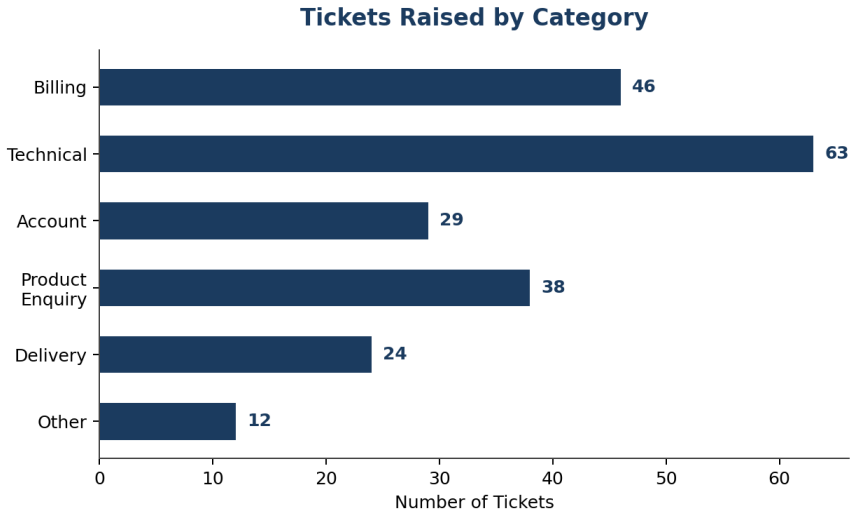


Figure 7.2: Tickets raised by category

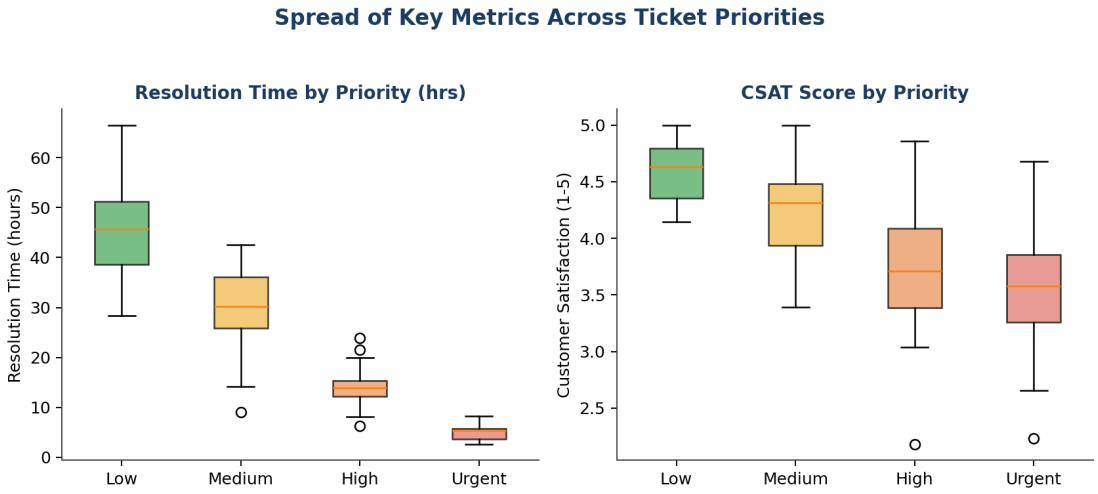


Figure 7.3: Spread of resolution time and CSAT score across ticket priorities

Figure 7.4: Resolution time vs. customer satisfaction, coloured by priority

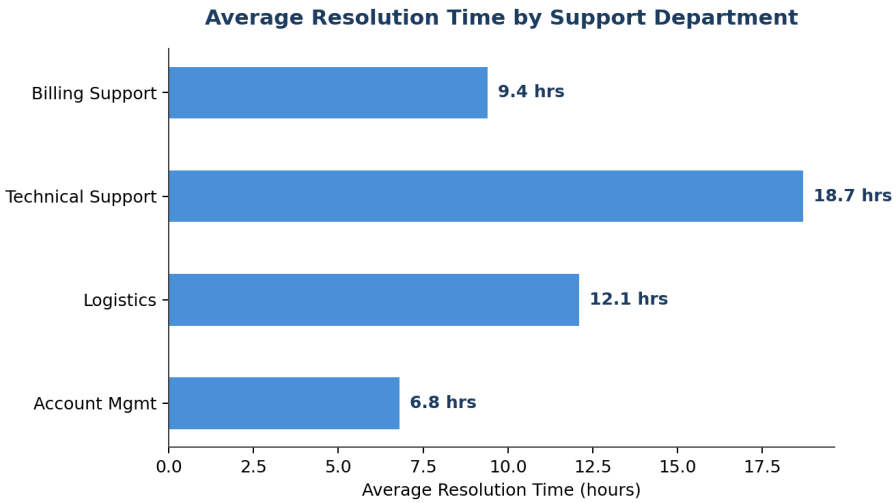


Figure 7.5: Average resolution time by support department

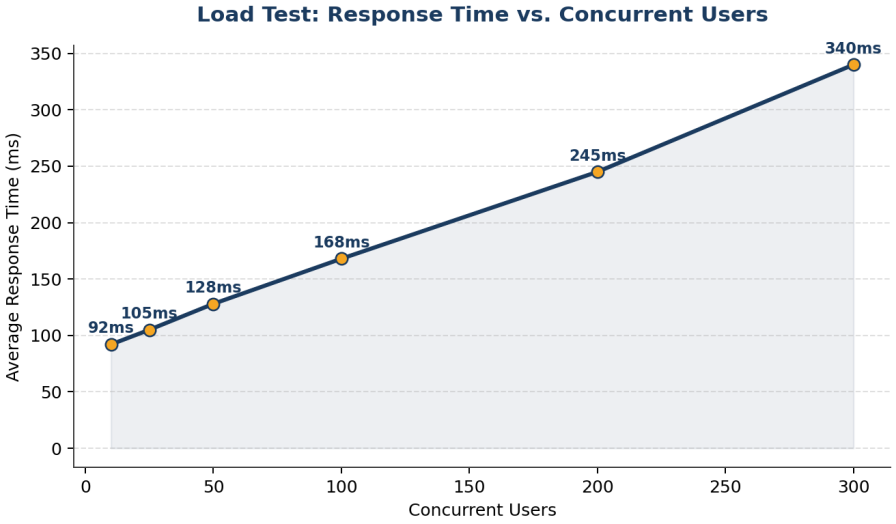


Figure 7.6: Load test — response time vs. concurrent users

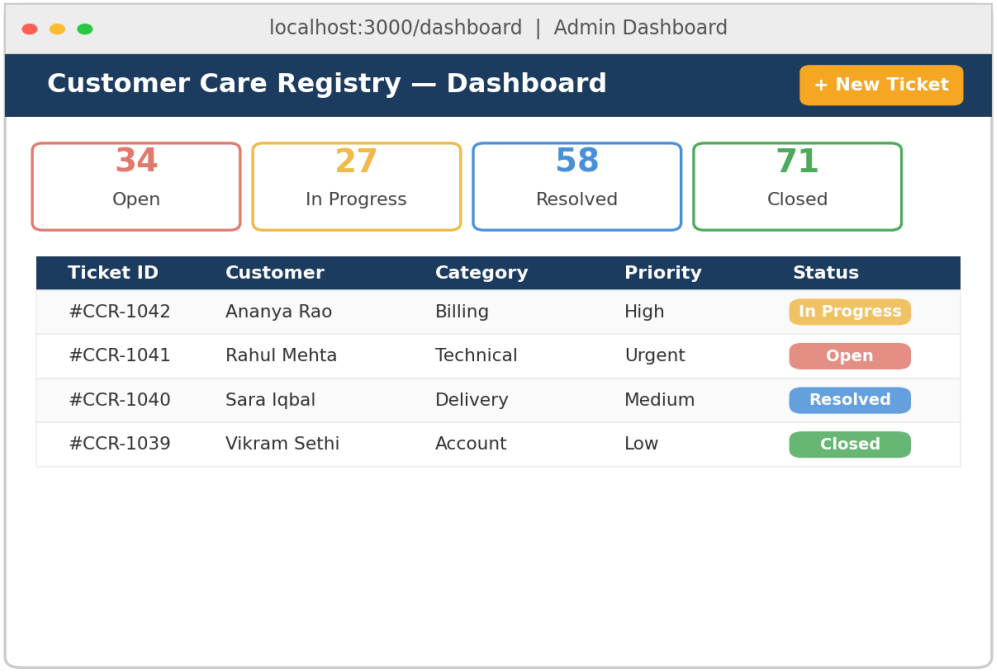


Figure 7.7: React admin dashboard — ticket overview and management

8. Advantages & Disadvantages

Advantages	Disadvantages
Provides a single, centralised record of every customer interaction and its history.	Requires reliable internet/server availability to access the deployed application.
Reduces duplicate work and missed follow-ups through automated ticket assignment.	Effectiveness depends on agents consistently updating ticket status.
Role-based dashboards give managers real-time visibility into support performance.	Does not yet support real-time chat; customers must refresh for updates.
Simple, accessible React interface usable by non-technical customers and agents.	Requires periodic scaling of the database and API as ticket volume grows.
Easily extensible to add new ticket categories, SLA rules or notification channels.	Initial setup requires configuring MongoDB Atlas, JWT secrets and email credentials.

9. Conclusion

This project successfully demonstrates the use of the MERN stack to build a Customer Care Registry that centralises customer complaints, requests and feedback into a single, trackable system. Tickets move through a clear lifecycle — submission, validation, assignment, resolution and feedback — with role-based dashboards giving customers, agents and managers the visibility they need.

By combining a React front end, an Express/Node REST API secured with JWT authentication, and a MongoDB database, the system offers a scalable, consistent, and rapid alternative to manual complaint tracking. This can meaningfully support businesses in improving response times, identifying recurring issues, and strengthening customer trust and loyalty.

10. Future Scope

- Add real-time updates using WebSockets (Socket.IO) so ticket status changes instantly for all users.
- Integrate an AI-powered chatbot for first-response triage before a ticket reaches an agent.
- Introduce SLA (Service Level Agreement) tracking with automatic escalation for overdue tickets.
- Enable batch import/export of tickets via CSV for reporting and analytics.
- Add multi-language support for customers across different regions.
- Build a native mobile app (React Native) for agents to manage tickets on the go.
- Integrate analytics dashboards with predictive insights on ticket volume and staffing needs.

11. Appendix

The links and source code below supplement the Appendix section of this report.

GitHub Repository: https://github.com/your-username/customer-care-registry-mern

Live Demo: https://customer-care-registry.onrender.com

Source Code

Representative code excerpts from the project's implementation are provided below, covering the MongoDB schema, Express controllers, JWT-protected routes, and the React front end. The complete, runnable source code is available in the GitHub repository linked above.

```
// a) Ticket Model & Schema (models/Ticket.js)
const mongoose = require('mongoose');

const ticketSchema = new mongoose.Schema({
  customerName: { type: String, required: true },
  email: { type: String, required: true },
  category: { type: String, enum: ['Billing', 'Technical', 'Account',
      'Delivery', 'Product Enquiry', 'Other'], required: true },
  priority: { type: String, enum: ['Low', 'Medium', 'High', 'Urgent'],
      default: 'Medium' },
  description: { type: String, required: true },
  status: { type: String, enum: ['Open', 'In Progress', 'Resolved',
      'Closed'], default: 'Open' },
  assignedAgent: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  feedbackRating: { type: Number, min: 1, max: 5 },
}, { timestamps: true });

module.exports = mongoose.model('Ticket', ticketSchema);

// b) Ticket Controller (controllers/ticketController.js)
const Ticket = require('../models/Ticket');

exports.createTicket = async (req, res) => {
  try {
    const { customerName, email, category, priority,
        description } = req.body;
    const ticket = await Ticket.create({
      customerName, email, category, priority, description,
    });
    res.status(201).json({ success: true, ticket });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message });
  }
};

exports.updateTicketStatus = async (req, res) => {
  const { status } = req.body;
  const ticket = await Ticket.findByIdAndUpdate(
    req.params.id, { status }, { new: true }
  );
  if (!ticket) return res.status(404).json({ message: 'Not found' });
  res.json({ success: true, ticket });
};

// c) Ticket Routes with JWT Auth Middleware (routes/ticketRoutes.js)
const express = require('express');
const router = express.Router();
const { protect, authorize } = require('../middleware/auth');
const {
  createTicket, getTickets, updateTicketStatus,
} = require('../controllers/ticketController');

router.post('/', createTicket);
router.get('/', protect, getTickets);
router.put('/:id/status', protect, authorize('agent', 'manager'),
    updateTicketStatus);

module.exports = router;

// d) React Ticket Form Component (src/components/TicketForm.jsx)
import { useState } from 'react';
```

```
import axios from 'axios';

export default function TicketForm() {
  const [form, setForm] = useState({
    customerName: '', email: '', category: 'Billing',
    priority: 'Medium', description: '',
  });

  const handleSubmit = async (e) => {
    e.preventDefault();
    const res = await axios.post('/api/tickets', form);
    alert(`Ticket #${res.data.ticket._id} created successfully`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input placeholder="Customer Name" value={form.customerName}
        onChange={(e) => setForm({ ...form, customerName: e.target.value })} />
      <textarea placeholder="Describe your issue" value={form.description}
        onChange={(e) => setForm({ ...form, description: e.target.value })} />
      <button type="submit">Submit Ticket</button>
    </form>
  );
}
```